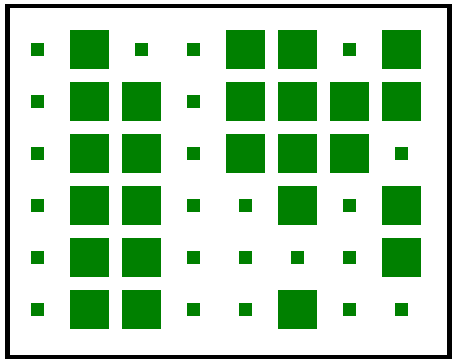
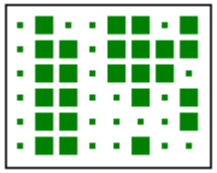




Class 8
Make and Flex

CS 330
Programming Languages
And Compilers

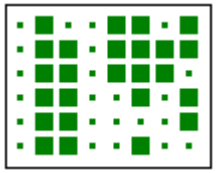
Obtain the Lab 2 Starting Code



- Open a terminal and change directory into your **`cs330-IndividualLabWork`** directory
- Use git to obtain the lab 2 code:

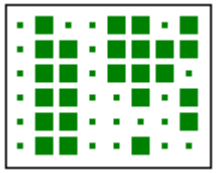
```
git pull
```

- The directory **`lab2-Flex-Introduction`** should be added to your project
  - Contains a Makefile, Flex configuration files, and C source/header files
- You will be writing additional Flex configurations and C programs for your lab assignment this week



Make

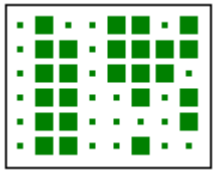
Lots of Source Files



Medium and large software projects utilize many files and tools that need to be coordinated

- Source code → compiled
- Header files → included, compiled
- Assets (images, sounds) → embedded
- Metadata (generate code) → converted, compiled
- Configuration → embedded
- Static data → embedded
- Testing → compiled, executed, reported

Source Files for Compiler



`b-minor-scanner.l` → Metadata for Flex (lexemes → tokens)

`b-minor-parser.y` → Metadata for Bison (grammar)

`ast_library.c/.h` → manages AST

`hash_table.c/.h` → manages symbol table entries

`symbol.c/.h` → manages symbols

`scope.c/.h` → manages symbol scope

`type_check.c/.h` → validates types across AST

`static_semantic_tests.c/.h` → testing

`code_gen.c/h` → converts AST to assembly code

`code_gen_util.c/.h` → manages code gen

main C source → kicks of compilation

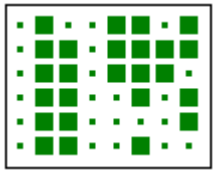
`b_minor_library.c/.h` → library functions (linked)

How to coordinate the re-build process when a file is changed?

What other files are impacted?

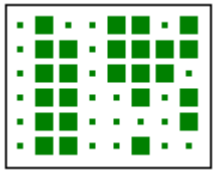
What steps need to be executed to generate source code, compile, and link?

What is `make` (from GNU Documentation)



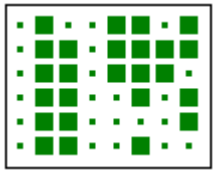
- A tool which controls the generation of executables and other generated files based on a configuration file and the program's source files
- `make` gets its configuration for building your program from a file called the **`Makefile`**, which lists each of the non-source files and defines how to generate it from other files
  - When you write a program, you write a `Makefile` for it to use `make` to build, test, and install the program

Capabilities of `make` (from the docs)



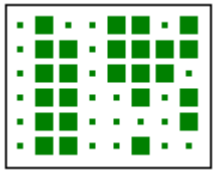
- `make` figures out **which files it needs to update**, based on which source files change.
  - Determines the **proper order** for updating files
- If you change source files and run `make`, it **updates non-source files** that depend on the changed source files
- `make` is **not limited to any particular language**. For each non-source file in the program, the `Makefile` specifies shell commands to compute it
  - For example: **compiler** to produce an object file, **linker** to produce an executable, **Flex** to generate a parser, **LaTeX** to format documentation

Configuration File (`Makefile`)



- `make` reads a file named **`Makefile`** in the current directory and takes its configuration information from there
- `Makefile` consists of a series of **targets** defining the commands to be used to build the program
- `make` also supports **variables** in the configuration file to improve readability and reduce redundant target definitions
- First target is the **default target**
  - If you run **`make`** with no arguments, the default target will be built

Example `Makefile`



```
PROGRAM=hello-world
OBJS=hello.o main.o
SRCS=hello.c main.c hello.h
main.h
CC=cc

all:$(PROGRAM)

hello.o: hello.c hello.h
    $(CC) -c -o $@ hello.c

main.o: main.c main.h hello.h
    $(CC) -c -o $@ main.c

$(PROGRAM): $(OBJS)
    $(CC) $(OBJS) -o $@ -lm

clean:; rm -f $(OBJS) hello-
world

touchsrc:; touch $(SRCS)

rebuild: touchsrc all
```

Variables

First target is default

Target consists of a name, optional list of dependencies, and optional command

Format of target

name: dependencies
optional command

or

name:; command

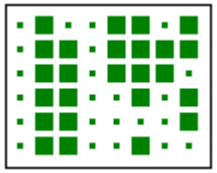
Additional Syntax

$(VAR) means use variable value

$@ means use target name

; means there are no dependencies

Using make



```
$ make
cc -c -o hello.o hello.c
cc -c -o main.o main.c
cc hello.o main.o -o hello-world -lm
```

- No target, default target used
- No dependencies met, all built

```
$ make
cc -c -o hello.o hello.c
cc hello.o main.o -o hello-world -lm
```

- No target, default target used
- `hello.c` edited, impacted dependencies rebuilt

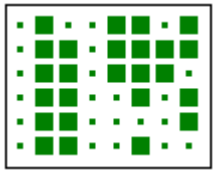
```
$ make hello.o
cc -c -o hello.o hello.c
```

- `hello.o` target
- `hello.c` only rebuilt if necessary

```
$ make clean
rm -f hello.o main.o hello-world
```

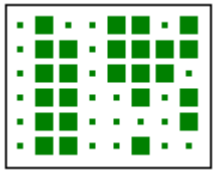
- `clean` target
- Idiomatic target defined to remove all generated files (e.g. everything except human-authored files)

Sample from Compiler `Makefile`



```
YFLAGS=-v --defines=tokens.h
YACC=bison
PROGRAM=b-minor-compiler
OBJS=b-minor-parser-ast.o b-minor-scanner.o ast_library.o ast_printer.o main.o ...
SRCS=b-minor-scanner.c b-minor-parser-ast.c ast_library.c ast_printer.c main.c ...
CC=cc
all:$(PROGRAM)
ast_library.o: ast_library.c ast_library.h
    $(CC) -c -o $@ ast_library.c
ast_printer.o: ast_printer.c ast_printer.h
    $(CC) -c -o $@ ast_printer.c
b-minor-parser-ast.c: b-minor-parser.y
    bison $(YFLAGS) --output=$@ b-minor-parser.y
b-minor-compiler:$(OBJS)
    $(CC) $(OBJS) -o $@ -lm
clean:; rm -f $(OBJS) b-minor-program generated_assembly_code.s ...
b-minor-program: generated_assembly_code.s b_minor_library.c
    $(CC) -o $@ generated_assembly_code.s b_minor_library.c
```

Example Use of `make`



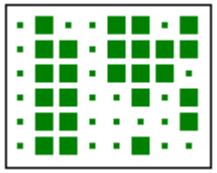
```
$ make
flex -C -o b-minor-scanner.c b-minor-scanner.l
cc    -c -o b-minor-scanner.o b-minor-scanner.c
cc b-minor-parser-ast.o b-minor-scanner.o ast_library.o ast_printer.o main.o static_semantic_tests.o
hash_table.o scope.o symbol.o type_check.o code_gen_util.o code_gen.o -o b-minor-compiler -lm

$ ./b-minor-compiler b-minor-test-programs/good3-add.b
>=================>(lex) Ignored single line comment
>=================>identifier get_int_from_keyboard
decl>
 name:get_int_from_keyboard  type:6 (TYPE_FUNCTION) subtype:3 (TYPE_INTEGER)
 symbol>
  kind:2 (SYMBOL_GLOBAL)
  type:6 (TYPE_FUNCTION) subtype:3 (TYPE_INTEGER)
CODEGEN>global get_int_from_keyboard of kind TYPE_FUNCTION
Assembler code written to generated_assembler_code.s

$ make b-minor-program
cc -o b-minor-program generated_assembler_code.s b_minor_library.c

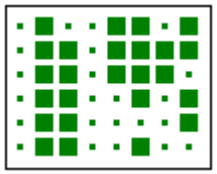
$ ./b-minor-program
Adding 2 numbers
First number? 6
Second number? 9
The sum of 6 and 9 is 15
```



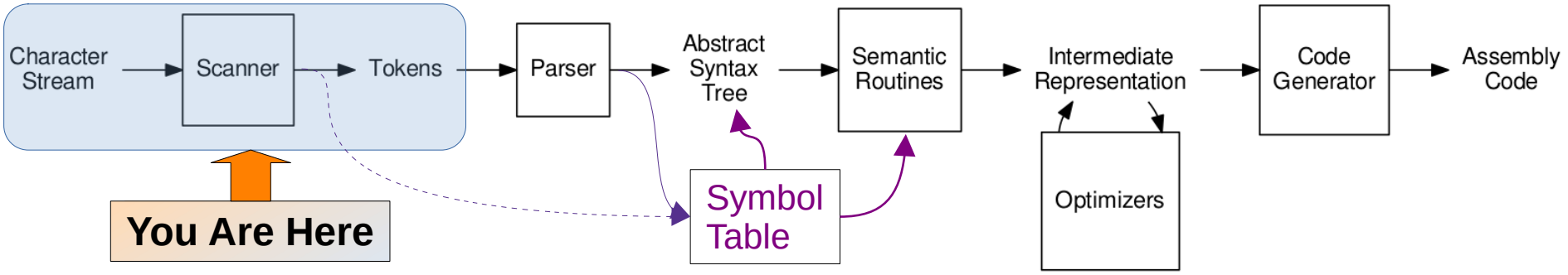


Flex Introduction

Compilation Process Overview

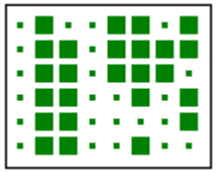


Thain, D. (2020). Introduction to compilers and language design.



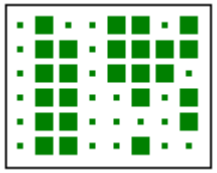
- **Scanner (Lexical analyzer) – tokenizes**
- Parser (Syntax analyzer) – lexical units syntax check
  - Creates symbol table and AST
- Semantic Routines – static semantics analysis/check
- Intermediate Rep – lower level language (optional)
- Optimizer – performance improvements (optional)
- Code Generator – outputs assembly lang (in our case)

Scanner Operation (Lexical Analysis)



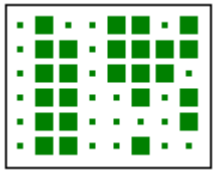
- The scanner looks for patterns of characters in the input
  - Regular expressions are a natural choice for defining the patterns
- DFAs implemented in code are faster than using a regex engine
  - DFA implementations can be generated from regular expressions

Flex: a Tool to Generate a Scanner



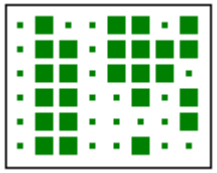
- Flex reads regular expressions and generates C code implementing equivalent DFAs
- Flex associates each regular expression with C code which is executed when a pattern is matched: **action**
  - Flex generates DFAs in C which incorporate the action C code we define

`yylex()` Runs Lexical Analysis



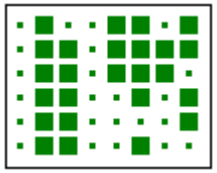
- Flex generates a “hook” function to run the scanning process: **`yylex()`**
- If the scanner returns tokens, then `yylex()` must be called repeatedly to fetch subsequent tokens
  - Tokens are represented as **integers**
  - `yylex()` **returns** the next token
- `yylex()` **returns 0** (zero) when the **end of the input** file is reached

Flex's Input File



- Flex reads a configuration file containing the regular expressions, actions, and optional configuration information
  - Flex generates a C program: **`lex.yy.c`**
- Flex's configuration file contains three sections, separated by **%%** (two percent signs):
  1) Declarations (optional)
  2) Rules (REs and actions)
  3) Additional C code (optional)

Example Flex Configuration File: wc.l



```
/* Count words, lines, and characters in a file */

%{
  int chars = 0;
  int words = 0;
  int lines = 0;
%}

%option noyywrap

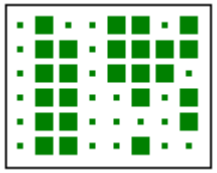
%%

[a-zA-Z]+  { words++; chars += strlen(yytext); }
\n         { chars++; lines++; }
.          { chars++; }

%%

int main(int argc, char **argv) {
  yylex();
  printf("%8d %8d %8d\n", lines, words, chars);
  return 0;
}
```

Example Flex Configuration File: wc.l



```
/* Count words, lines, and characters in a file */

%{
  int chars = 0;
  int words = 0;
  int lines = 0;
%}

%option noyywrap

%%

[a-zA-Z]+   { words++; chars += strlen(yytext); }
\n          { chars++; lines++; }
.           { chars++; }

%%

int main(int argc, char **argv) {
  yylex();
  printf("%8d %8d %8d\n", lines, words, chars);
  return 0;
}
```

Denotes C code block to be inserted into generated scanner *prior* to actions

Options modify flex defaults

Section separator: %%

Section separator: %%

Definitions Section

Declarations and Options

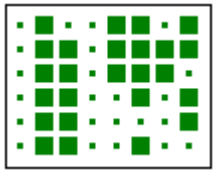
Rules Section

Regex and actions

User Code Section

Additional C Code

Generating the Scanner C Code

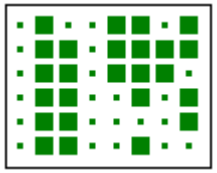


- The **`flex`** command is used to read the flex configuration file and generate C code

```
flex wc.l
```

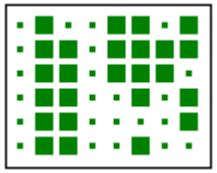
- If no `main()` is provided in the Flex configuration, compilation requires using the `-c` option
  - We do not include `main()` when using flex as part of a compiler
  - If standalone, with main: **`cc lex.yy.c`**
  - If part of a compiler, no main: **`cc -c lex.yy.c`**

Using the Generated Scanner



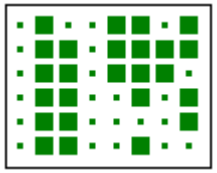
- The `wc.l` example includes `main()`, so we compile for standalone execution: **`cc lex.yy.c`**
  - By default, the generated scanner reads characters from the standard input
  - Remember that **`yylex()`** runs the scanning process: our `wc.l` calls `yylex()` in `main()`
- Run the program: **`./a.out`**
  - Type words, newlines, and other characters
  - Use `[ctrl]-D` to signal end-of-file (no more input) to the program and see the output

Returning Token Values



- When using Flex as part of the compiler toolchain, its role is to map the input characters (grouped into lexemes) to tokens
  - ***Actions <u>return</u> tokens***
- The parser will consume tokens, calling `yylex()` repeatedly to get each token in succession
  - We'll use Flex to build a simple calculator to experience how token processing works

calc_skeleton.l: Decl. and Options



```
/* recognize tokens for a calculator and print them out */

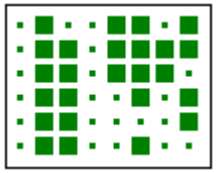
%{
  enum yytokentype {
    NUMBER = 258,
    ADD = 259,
    EOL = 998,
    EXIT = 999
  };

  int yylval;
%}

%option noyywrap

%%
```

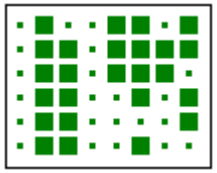
calc_skeleton.l: Regex and Actions



```
"+" { return ADD; }
exit { return EXIT; }
[0-9]+ { yylval = atoi(yytext); return NUMBER; }
\n  { return EOL; }
[ \t] { /* ignore whitespace - note no returned
    value,yylex() will keep looking for a token */ }
.   { printf("Warning: character %c skipped\n",
                                        *yytext); }

%%
```

calc_skeleton.l: Additional C Code

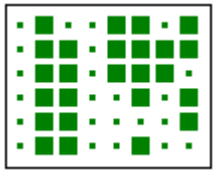


```
int main(int argc, char **argv) {
  int token;

  while (token = yylex()) {
    printf("Token Id:%d", token);
    if (token == NUMBER) {
      printf(" = %d\n", yylval);
    } else {
      printf("\n");
    }
  }

  return 0;
}
```

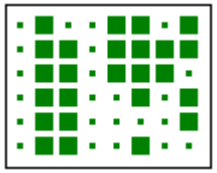
Using the calc_skeleton.l Scanner



```
flex calc_skeleton.l
```

- We included a `main()` so compile for standalone execution: **`cc lex.yy.c`**
- Run the program: **`./a.out`**
  - Type some numbers and operators, pressing enter at the end of each expression
  - Check that the token values being output are as expected
  - Use `[ctrl]-D` to end the program

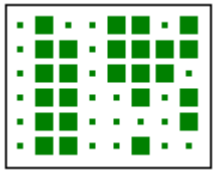
More Compiler Foreshadowing



- How does the calculator skeleton get the value of a number back to the code calling `yylex()`?
  - NOT the token value, ***the literal's value?***
- How could the same concept be used to deal with situations where the value we care about is sometimes a string and sometimes a number?
- We will use this technique so that our parser has access to values of literals and names of variables

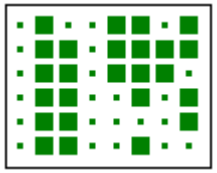


Implement Addition and Subtraction

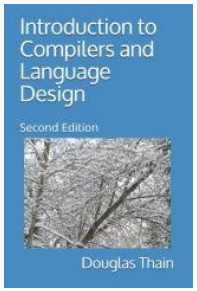


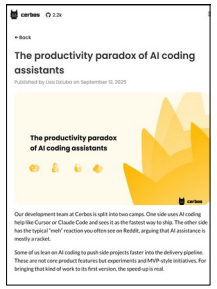
- We will implement addition and subtraction using Flex: **`calc_implementation.l`**
- We'll place the operational code (logic) in a separate file: **`add_sub.c`** and **`add_sub.h`**
- To build the calculator, we'll use Flex to generate the scanner, then compile the scanner and operational code, and finally link the object files to create the executable
- We won't use **`make`** yet, but you are free to create a `Makefile` to manage the process as part of your lab for **extra credit**

Homework



- Monday (9/28):
  - Lab 1 Due: C Coding (Team and Individual)
- Friday (10/2):
  - Compilers book Chapter 4
  - LLM: The productivity paradox…





- Sunday (10/4)
  - Journal entry for week 4
    - **2-1 Rule** (minimum 2 paragraphs, 1 quiz question)
- Monday (10/5):
  - Lab 2 Due: Flex & Make (Team and Individual)